# impl PrivacyProfile::to_tradeoff

Michael Shoemate

May 2, 2026

This proof resides in **"contrib"** because it has not completed the vetting process.

## 1 Hoare Triple

### Precondition

#### Compiler-verified

- Receiver `self` of type `PrivacyProfile`

#### Caller-verified

The receiver is a valid privacy profile.

### Pseudocode

```
def privacyprofile_to_tradeoff(I: Intent, profile: PrivacyProfile) -> TradeoffCurve:
    if profile.epsilon_grid is not None:  #
        eps_samples = list(profile.epsilon_grid)
    else:
        eps_hi = EPS_MAX_START
        last_delta = profile.delta(eps_hi)
        while last_delta > 1e-12 and is_finite(eps_hi) and eps_hi < 1e6:  #
            eps_hi *= 2.0
            delta = profile.delta(eps_hi)
            if delta >= last_delta:
                break
            last_delta = delta
        eps_samples = [eps_hi * i / EPS_GRID_POINTS for i in range(EPS_GRID_POINTS + 1)]  #

    supports = []
    last_delta = None

    for eps in eps_samples:  #
        delta = profile.delta(eps)
        delta_exact = RBig.try_from(delta)

        # If delta is unchanged and epsilon increased, the new support is pointwise worse.
        if last_delta == delta_exact:  #
            continue

        tradeoff = approximate_to_tradeoff(I, eps, delta)
        supports.append(tradeoff)  #
        last_delta = delta_exact

```

```
30     def tradeoff_curve(alpha: float) -> float:
31         if not (is_finite(alpha) and 0.0 <= alpha <= 1.0):
32             raise ValueError("alpha ({alpha}) must be a finite number in [0, 1]")
33 
34         if alpha == 0.0:  #
35             return 1.0
36         if alpha == 1.0:  #
37             return 0.0
38 
39         alpha = RBig.try_from(alpha)
40         best = rbig(0)
41 
42         for support in supports:  #
43             best = best.max(support(alpha))
44 
45         result = round_to_f64(best, Opp[I])  #
46         return result.clamp(0.0, 1.0)  #
47 
48     return TradeoffCurve(tradeoff_curve)
```

## Postcondition

**Theorem 1.1.** The pseudocode returns a valid tradeoff curve $\beta(\alpha) = \max_i \beta_i(\alpha)$ where each $\beta_i$ is conservative with respect to the intent: for `Reporting` it is a lower bound on the sampled $f$-DP tradeoff, and for `Calibration` it is an upper bound on the sampled $f$-DP tradeoff.

*Proof.* Let $\mathcal{E} = \{\epsilon_1, \epsilon_2, \ldots, \epsilon_n\}$ be the set of sampled epsilon values (determined by lines 2–13). The choice of grid doesn't matter for privacy. Let $\delta_i = \delta(\epsilon_i)$ be the corresponding delta values from the profile. When no explicit grid exists, line 7 expands the upper endpoint until the profile is sufficiently small, the endpoint bound is reached, or the profile stops decreasing; this only determines the finite set of supports used in the approximation.

For each distinct $\delta_i$ encountered in the grid (loop starting at line 18, support appended at line 27), the algorithm constructs $\beta_i =$ `approximate_to_tradeoff`$(I, \epsilon_i, \delta_i)$ and appends it to the list `supports`. By the proof of `approximate_to_tradeoff`, each $\beta_i$ is a conservative lower bound for `Reporting` and a conservative upper bound for `Calibration`.

Line 23 skips any samples where $\delta(\epsilon) = \delta(\epsilon')$ for some $\epsilon < \epsilon'$ already processed. This is sound because for fixed $\delta$, increasing $\epsilon$ strictly weakens the approximate-DP constraint: the two affine branches of the approximate-DP tradeoff curve (described in `approximate_to_tradeoff`) are both pointwise smaller when $\epsilon$ increases. Thus the later support is redundant for both lower and upper conservative envelopes.

The returned tradeoff curve evaluates to $\beta(\alpha) = \max_i \beta_i(\alpha)$ for $\alpha \in (0, 1)$ (loop at line 42), with boundary values $\beta(0) = 1$ (line 34) and $\beta(1) = 0$ (line 36). The pointwise maximum of conservative lower bounds is again a conservative lower bound, and the pointwise maximum of conservative upper bounds is again a conservative upper bound over the sampled supports. Line 45 rounds the native return value downward for `Reporting` and upward for `Calibration`, preserving the intent.

Line 46 clamps the result to $[0, 1]$. For `Reporting`, raising a negative value to 0 remains below any valid tradeoff curve, and lowering a value above 1 is conservative. For `Calibration`, lowering a value above 1 to 1 remains above any valid tradeoff curve, and the maximum with zero prevents negative exact rational supports. □